

Генеративные рекомендательные системы

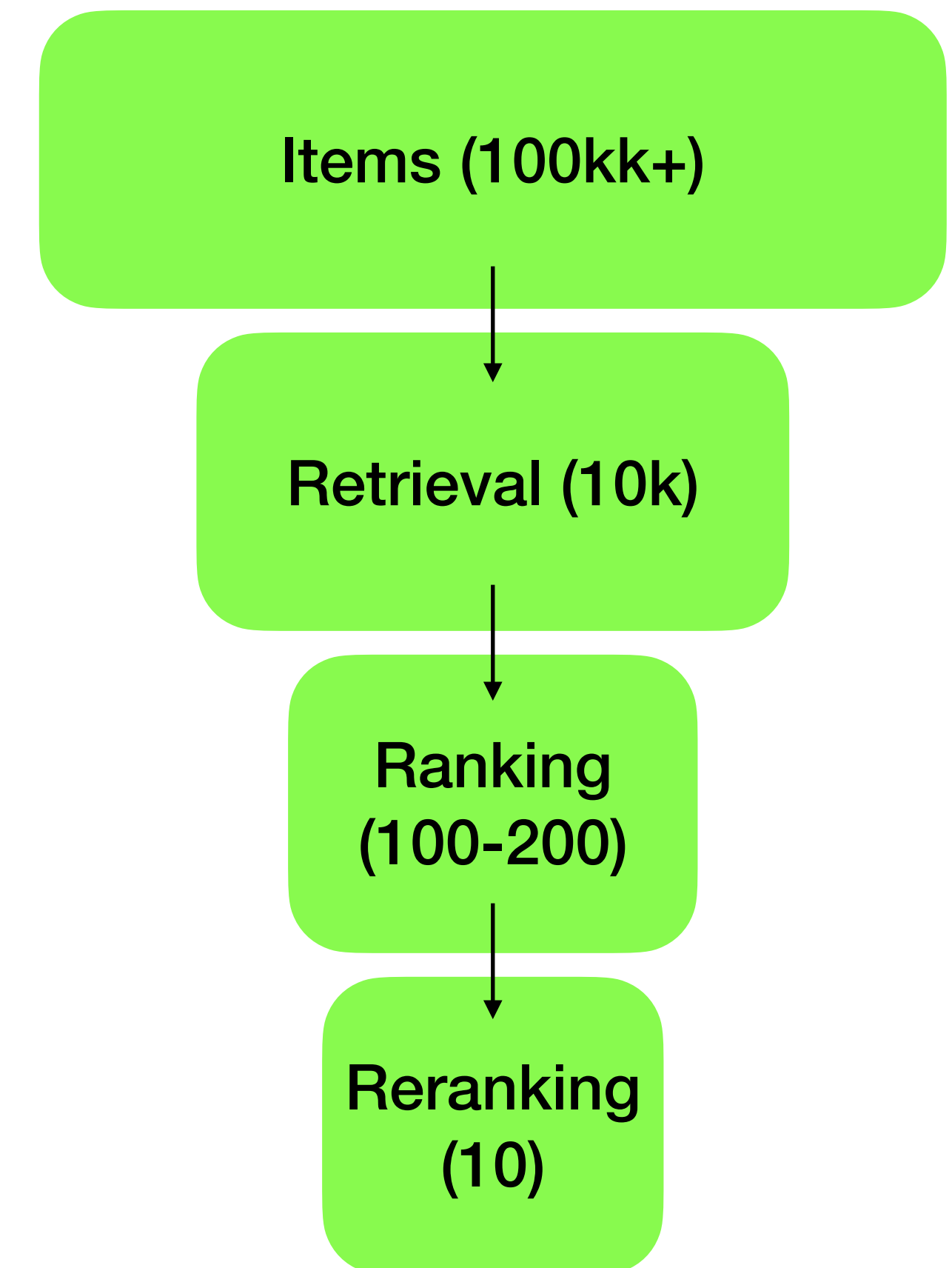
Осиновсков Илья, 20.04.2026. AI masters.

План лекции

- Мотивация — почему классические рекомендеры упёрлись в потолок
- Генеративный ретривал — TIGER и Semantic IDs
- Генеративный ранкинг — HSTU, LiGR
- Единые системы — OneRec, PLUM
- Scaling laws в рекомендациях
- Industrial deployment
- Сравнение и выводы

Как мы рекомендовали последние 10 лет

- Многоэтапная схема
- Технологии: Matrix Factorization, Two-Tower (DSSM, 2013), YouTube-версия 2019, Wide&Deep (Google, 2016), DLRM (Meta, 2019)
- Ключевые компоненты:
 - feature engineering
 - heavy embeddings tables
 - sparse features
 - sigmoid/softmax heads



Почему индустрия стала искать альтернативы

- Feature engineering bottleneck: тысячи hand-crafted фичей, годы работы команд
- Несогласованность стадий: ретривер и ранкер оптимизируют разные цели
- Потолок scaling: добавление параметров в DLRM даёт всё меньший прирост
- Изолированность от прогресса в NLP: LLM-архитектуры давали 100x улучшения, рекомендеры — проценты
- Cold start и длинный хвост: эмбединги плохо работают для новых айтемов

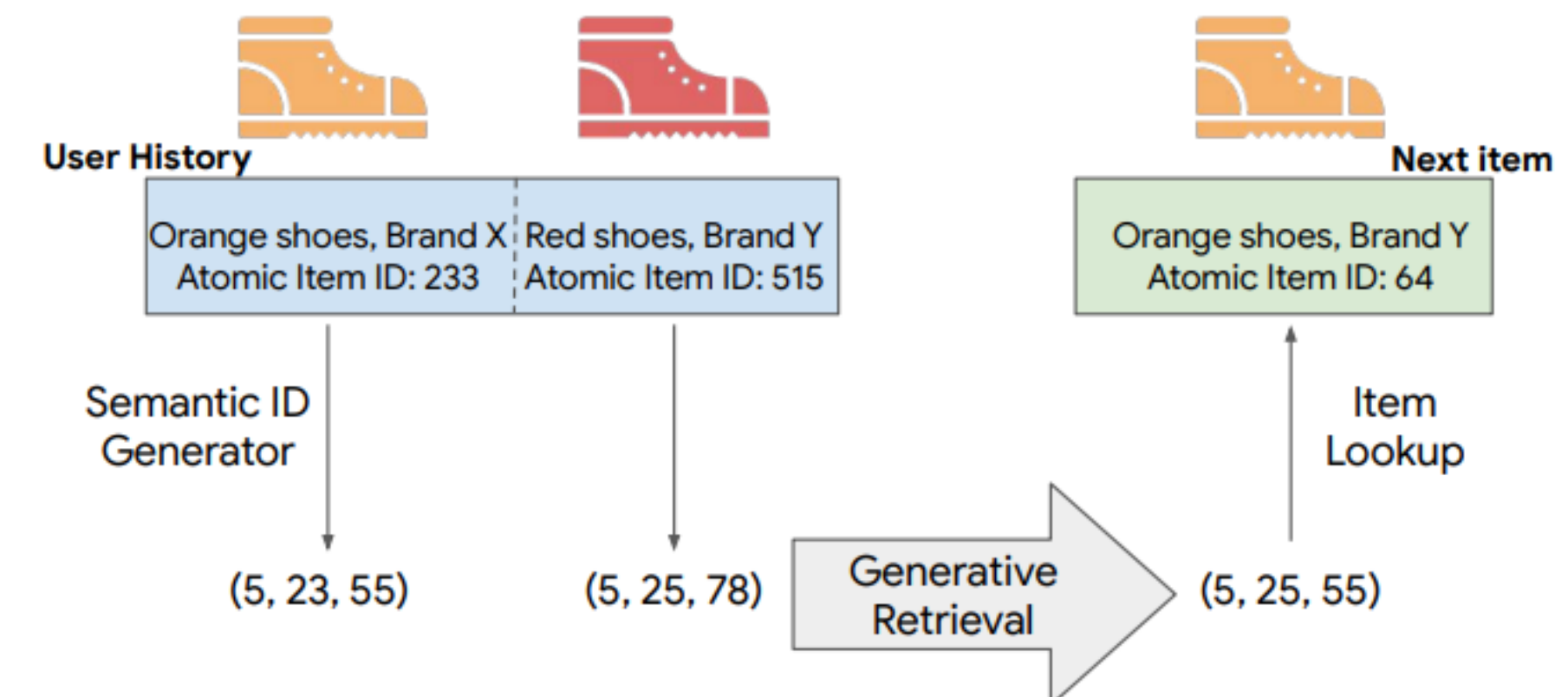
Сдвиг парадигмы — от матчинга к генерации

- Старое: $\text{user_emb} \cdot \text{item_emb} \rightarrow$ скор для каждого кандидата
- Новое: $P(\text{next_item} \mid \text{history})$ — авторегрессионная генерация следующего айтема
- Аналогия с LLM: user history = prompt, next item = next token
- Что это даёт: единая модель, transfer learning, scaling laws, zero-shot для НОВЫХ айТЕМОВ

Генеративный ретривал

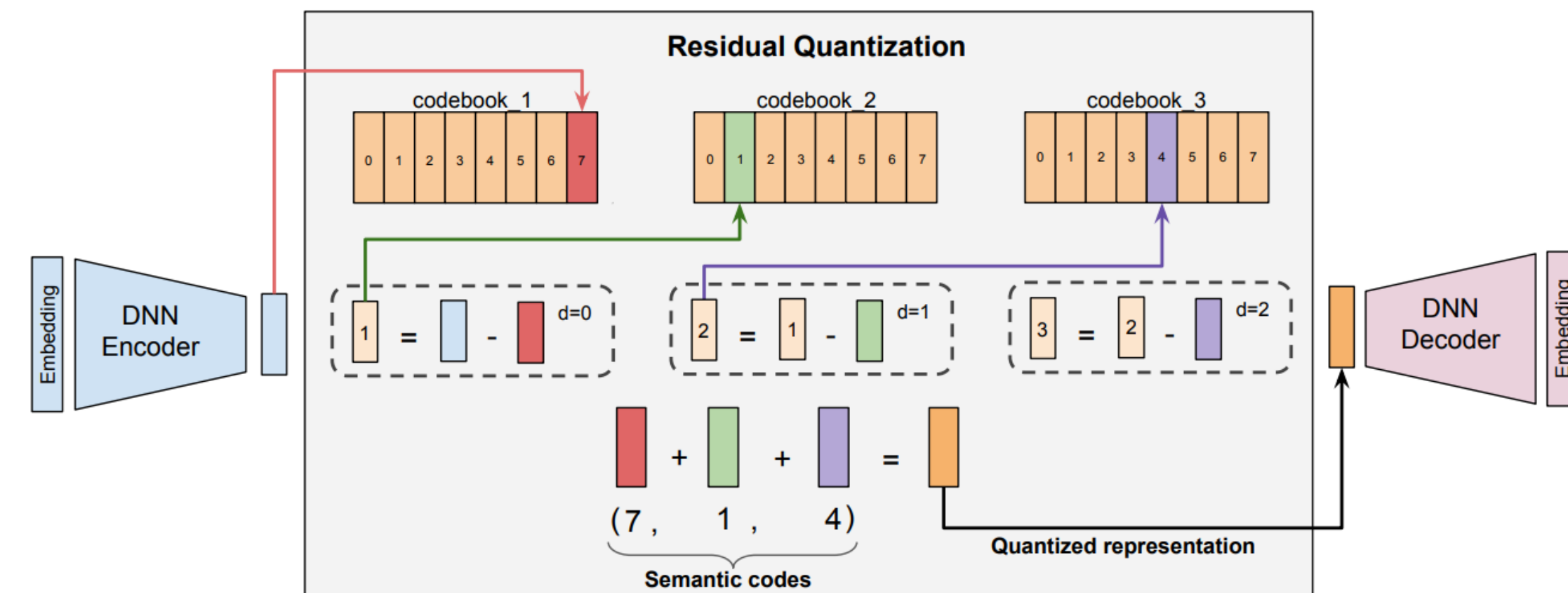
Recommender Systems with Generative Retrieval

- arXiv:2305.05065, Google, 2023
- Задача: заменить two-tower + ANN на один seq2seq трансформер
- Ключевое изобретение: Semantic IDs — дискретные коды вместо continuous embeddings
- Архитектура: T5-style encoder-decoder генерирует последовательность кодов



RQ-VAE для построения иерархических семантических кодов

- Шаг 1: content embedding айтема (из текстового описания через Sentence-T5)
- Шаг 2: Residual-Quantized VAE кодирует эмбеддинг в L уровней по K кодов
- Пример: айтем "iPhone 15 Pro Max" → (12, 127, 3, 91)
- Иерархичность: первый токен — грубая категория, последующие — уточнения



Как TIGER генерирует рекомендации

- Обучение: seq2seq cross-entropy на последовательностях Semantic ID
- Вход: [SID_1, SID_2, ..., SID_n] — история юзера
- Выход: SID_{n+1} — следующий айтем
- Инференс: beam search с constrained decoding (только валидные айтемные коды)
- Преимущества: zero-shot для новых айтемов (есть контент — есть SID), нет ANN-индекса
- Результаты: +29% Recall@5 на Amazon Beauty vs SASRec

Что TIGER НЕ решает

- Это только retrieval — ранкер всё ещё отдельный
- Semantic IDs строятся один раз — проблема drift для динамичных каталогов
- Небольшой масштаб экспериментов (академические датасеты)
- Не использует реальные сигналы взаимодействия (likes, watch time) — только последовательность

Semantic IDs стали промышленным стандартом

- TIGER (2023) — пионерская работа
- Meta HSTU (2024) — комбинация с ранкингом
- OneRec (Kuaishou, 2025) — RQ-VAE на мультимодальных эмбедингах
- PLUM (Google, 2025) — адаптация SID для pre-trained LLM

Как ещё можно строить Semantic IDs

Semantic IDs: RQ-VAE — не единственный путь

- **RQ-VAE** (TIGER, OneRec) — иерархические residual-коды, L уровней, обучаемые codebook'и через VAE-loss.
 - Плюсы: богатая иерархия, end-to-end обучение.
 - Минусы: нестабильность обучения, code collapse, сложность тюнинга
- **VQ-VAE** (van den Oord et al., 2017) — один уровень квантизации, «плоский» codebook.
 - Плюсы: проще, стабильнее.
 - Минусы: нет иерархии → хуже для холодного старта и constrained decoding, нужен больший codebook для той же выразительности
- **RQ-KMeans** (упоминается в работах Kuaishou и Pinterest) — residual-квантизация через обычный K-Means, без нейросети.
 - Плюсы: детерминированно, быстро, нет проблем обучения.
 - Минусы: не адаптируется к downstream-задаче, качество кодов зависит от качества входных эмбеддингов

Почему RQ-VAE ломается: проблема code collapse

- **Что это:** в процессе обучения большинство codebook-векторов перестают использоваться — активными остаются 5-10% кодов из тысячи
- **Почему происходит:** если в начале обучения какой-то код оказался ближе к большинству эмбеддингов — градиенты идут только к нему, он «притягивает» ещё больше векторов, остальные коды не обновляются
- **Последствия для рекомендера:** эффективный размер словаря меньше заявленного → разные айтемы получают одинаковые SID → коллизии → теряется смысл семантических кодов

Генеративный ранкинг

HSTU: трансформеры для рекомендаций в Meta

- arXiv:2402.17152, 2024
- Ключевой тезис: заменить DLRM на один большой трансформер, работающий на последовательности действий
- Не только "что кликнул", а "что кликнул, сколько смотрел, лайкнул, прокрутил, купил»
- Переформулировка ранкинга как sequence modeling

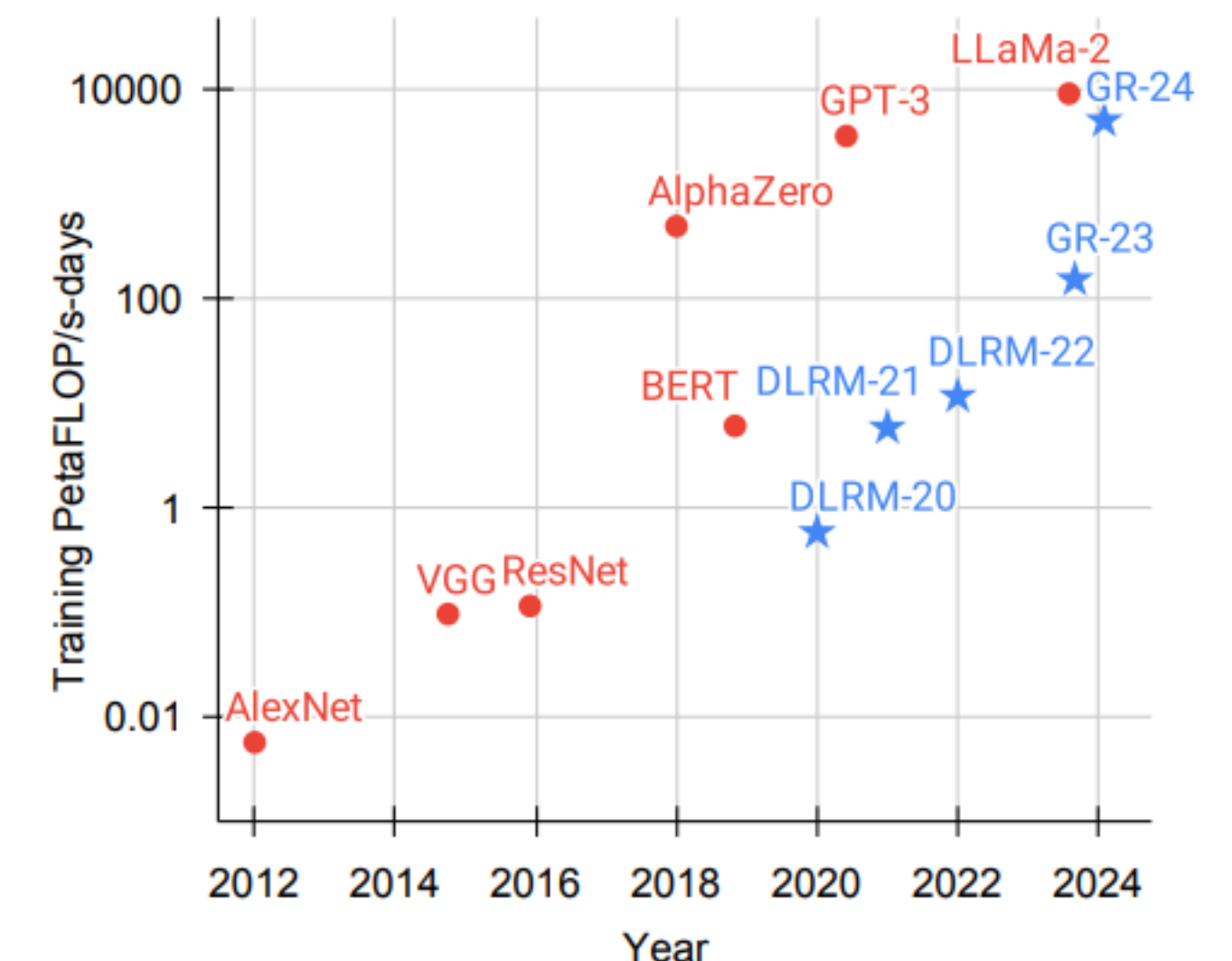
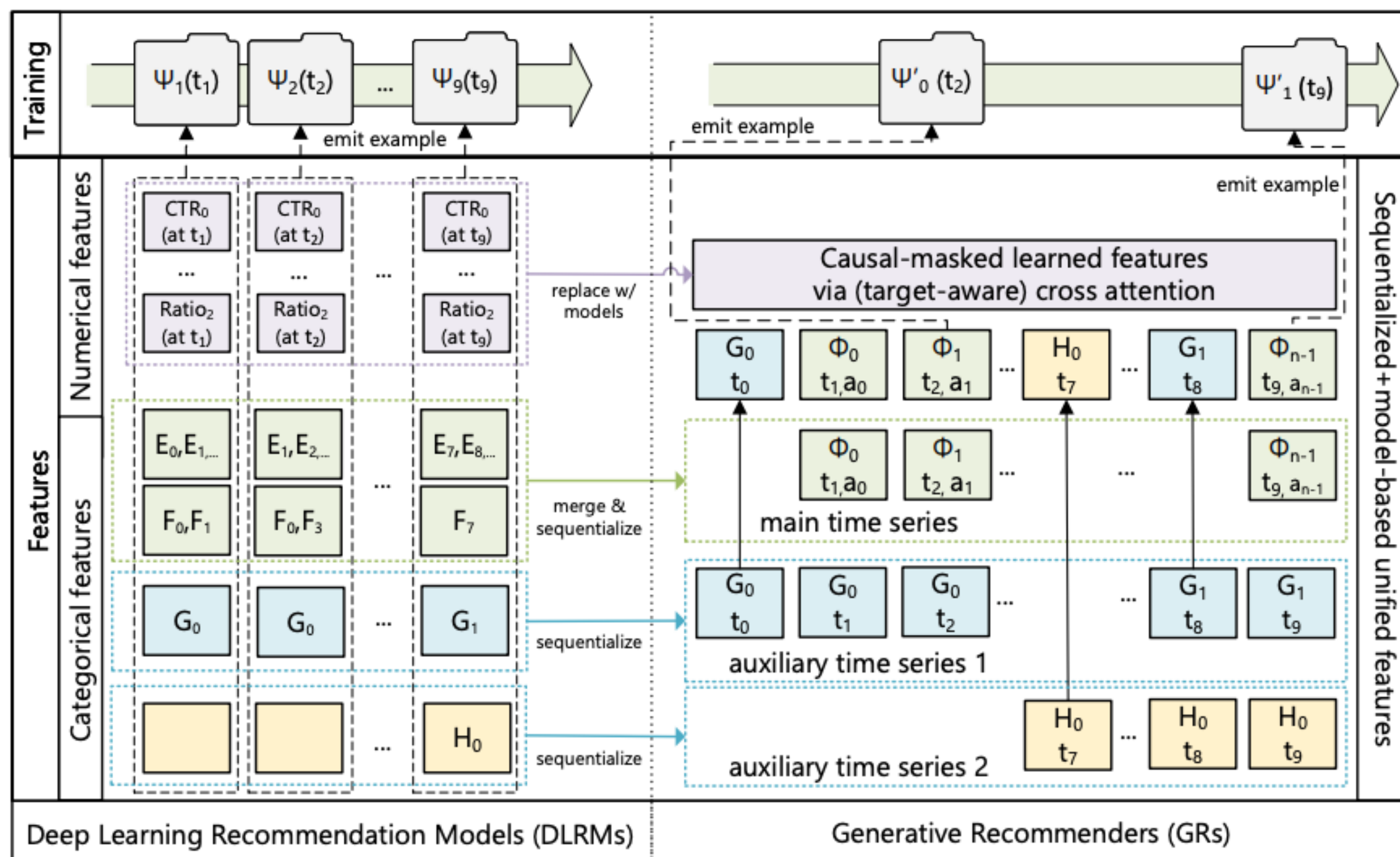


Figure 1. Total compute used to train deep learning models over the years. DLRM results are from (Mudigere et al., 2022); GRs are deployed models from this work. DLRMs/GRs are continuously trained in a streaming setting; we report compute used per year.

HSTU: трансформеры для рекомендаций в Meta



Hierarchical Sequential Transduction Unit

- Новый attention-блок: точечное умножение + SiLU, вместо стандартного softmax attention
 - softmax нормализует attention-веса в распределение (сумма = 1). Это удобно для языка, где важно «на какой токен смотреть», но теряется **абсолютная интенсивность** сигнала
- Объединение self-attention и pointwise проекций в один оператор (эффективно по памяти)
- Поддержка очень длинных последовательностей (до 8192 действий юзера)
- Обрабатывает ретривал и ранкинг в одной архитектуре (но с разными головами)

Отличие sequential от generative

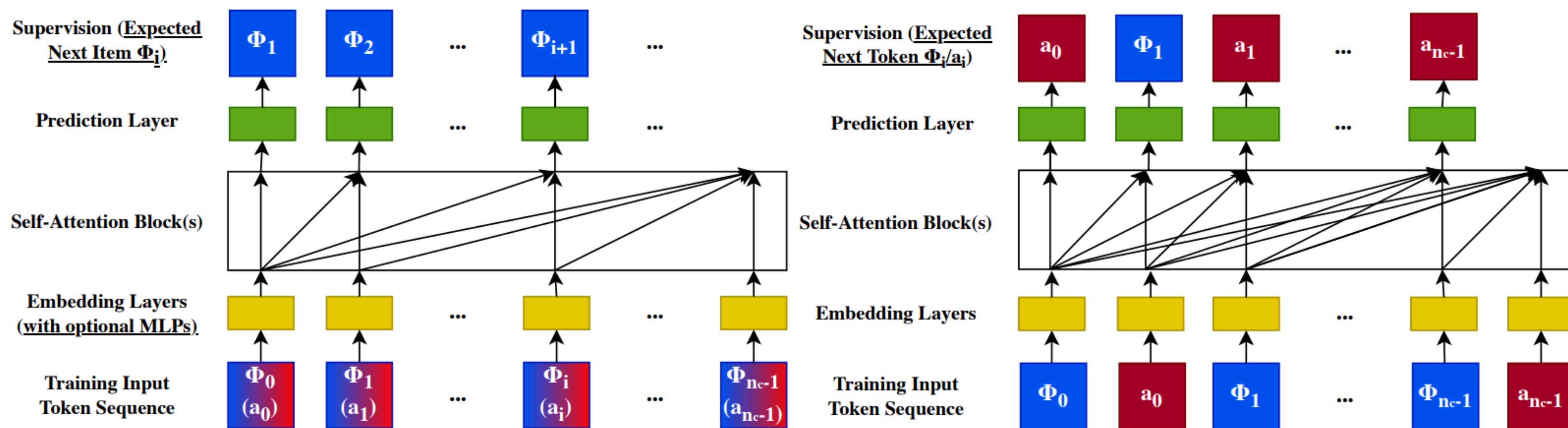


Figure 8. Comparison of traditional sequential recommenders (left) and Generative Recommenders (right). We illustrate sequential recommenders in causal autoregressive settings and GRs without contextual features to facilitate comparison. On the left hand side, the action types a_i s are either ignored or combined with item information Φ_i s using MLPs, before going into self-attention blocks.

Первые scaling laws в индустриальных рекомендациях

- Эксперименты на внутренних данных Meta (триллионы токенов действий)
- Параметры от 1.5M до 1.5B — качество растёт по степенному закону
- При N параметров качество $\propto N^\alpha$, $\alpha \approx 0.15-0.2$
- Запущено в production в Meta, замещает DLRM на нескольких поверхностях
- +12.4% engagement в A/B-тестах

Redefining Ranking for Scalable Impact

- arXiv:2502.03417, LinkedIn, 2025
- Задача: унифицированный ранкер для всех поверхностей LinkedIn (Feed, Jobs, People You May Know)
- Переход от тысяч hand-crafted фичей к трансформеру над последовательностью взаимодействий
- Результаты: +0.5% session time, +1.76% job applications

Redefining Ranking for Scalable Impact

Ключевые признаки GR подхода

- Последовательная формулировка задачи
- Action interleaving
- Semantic ids
- Но модель используется преимущественно как **ranking**, а не как retrieval через генерацию ID item'ов авторегрессионно. Кандидаты приходят извне из retrieval-слоя (LiNR).

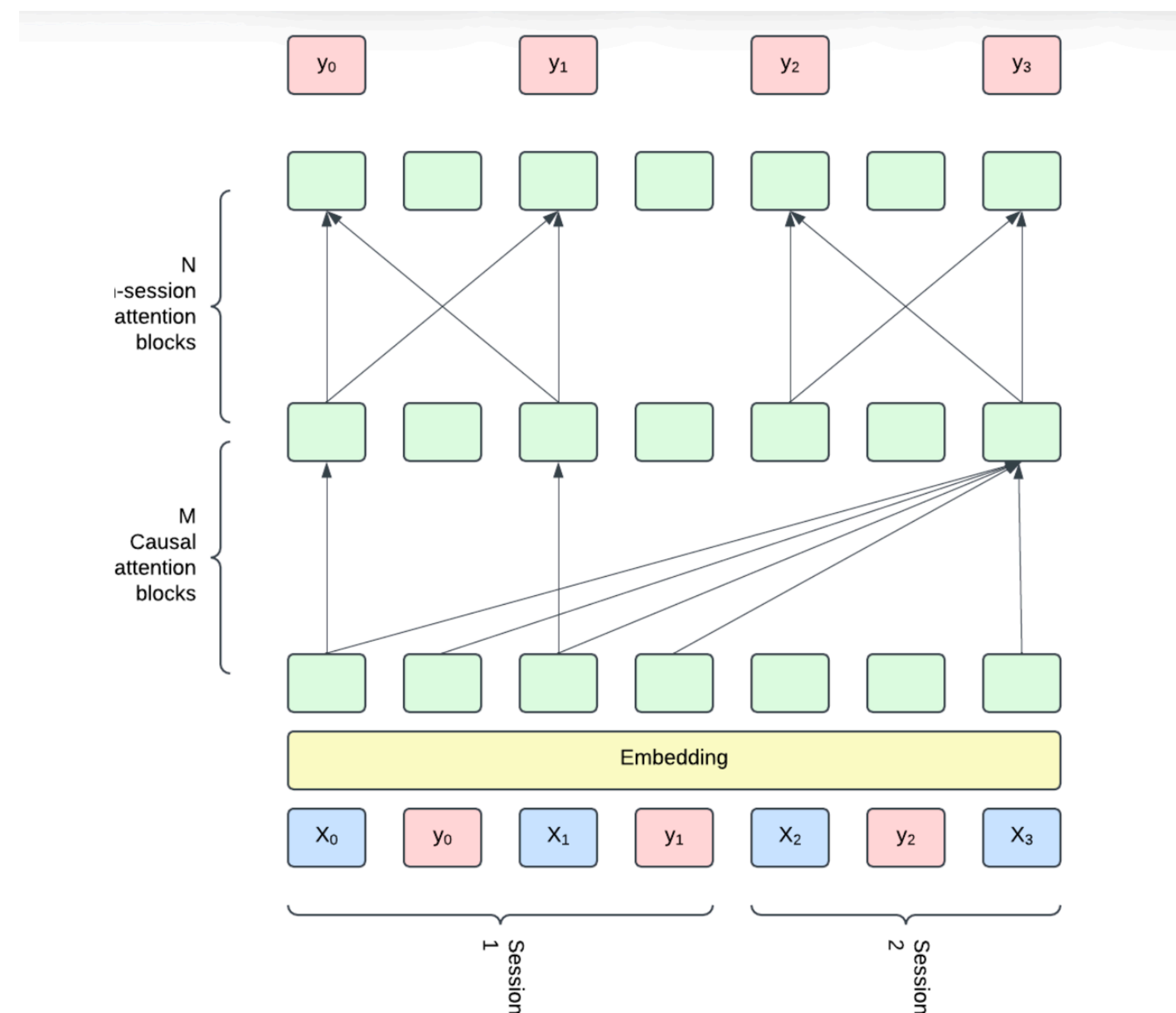


Figure 2: *LiGR* architecture combining historical attention for feature aggregation and in-session attention.

ARGUS: миллиардная рекомендательная модель

- arXiv:2507.15994, Яндекс, 2025
- AutoRegressive Generative User modeling for Search
- Первая опубликованная рекомендательная модель на 1B параметров в продакшене
- Обучение на исторических логах (триллионы событий)
- Pre-training → fine-tuning парадигма, как в LLM

ARGUS: миллиардная рекомендательная модель

Как устроен миллиардный рекомендер

- Decoder-only трансформер (GPT-style)
- Длина контекста — тысячи событий пользователя
- Multi-task objectives: предсказание следующего item, его типа действия, времени
- Дистилляция для production (учитель 1B → ученик 100M для serving)
- Infrastructure: обучение на сотнях H100, кастомные CUDA-ядра
- Схема pre-train → distill → deploy pipeline

ARGUS: миллиардная рекомендательная модель

Как устроен миллиардный рекомендатель

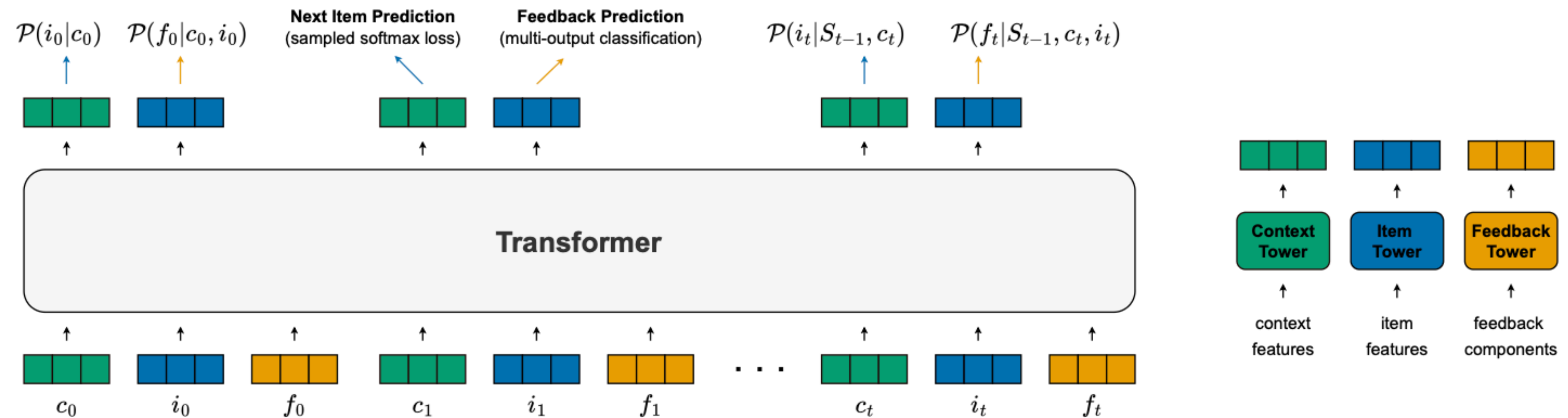


Figure 1: An overview of the pre-training architecture, which takes the user's context-item-feedback history as input to a transformer. Two parallel heads produce next-item predictions and feedback predictions, respectively.

End2end системы

Почему retrieval + ranking это неоптимально

- Двойное обучение: разные модели, разные лоссы
- Training-serving skew между стадиями
- Retrieval-ранкер цели не совпадают (Recall vs NDCG vs engagement)
- Два источника ошибок: хороший кандидат может не дойти до ранкера
- Мотивация объединения: end-to-end оптимизация целевой метрики

OneRec: unified retrieval и ranking

- arXiv:2502.18965, Kuaishou, февраль 2025
- Unifying Retrieve and Rank with Generative Recommender
- Одна модель заменяет классический cascade
- Iterative Preference Alignment — DPO-подобный тюнинг на сигналах engagement
- Развёрнуто на Kuaishou (800M DAU по видео)

OneRec: архитектура

Как устроена единая генеративная модель

- Encoder-decoder трансформер
- Айтемы закодированы мультимодальными Semantic IDs (видео + текст + аудио)
- Encoder: агрегирует историю взаимодействий пользователя
- Decoder: авторегрессионно генерирует последовательность SID для показа
- Session-wise генерация: за один forward pass — целая сессия рекомендаций

Iterative Preference Alignment (IPA)

RLHF для рекомендаций

- Проблема: cross-entropy на истории — проху-цель, не целевая метрика
- Решение: после SFT запустить DPO-like alignment на сигналах качества
- Сбор пар: (хорошая сессия, плохая сессия) по watch time / engagement
- Итеративность: модель → новые данные → алаймент → новые данные
- Результат: +1.6% watch time в A/B, 0.54% DAU uplift
- цикл SFT → Deploy → Collect preference data → DPO → Deploy

OneRec: архитектура

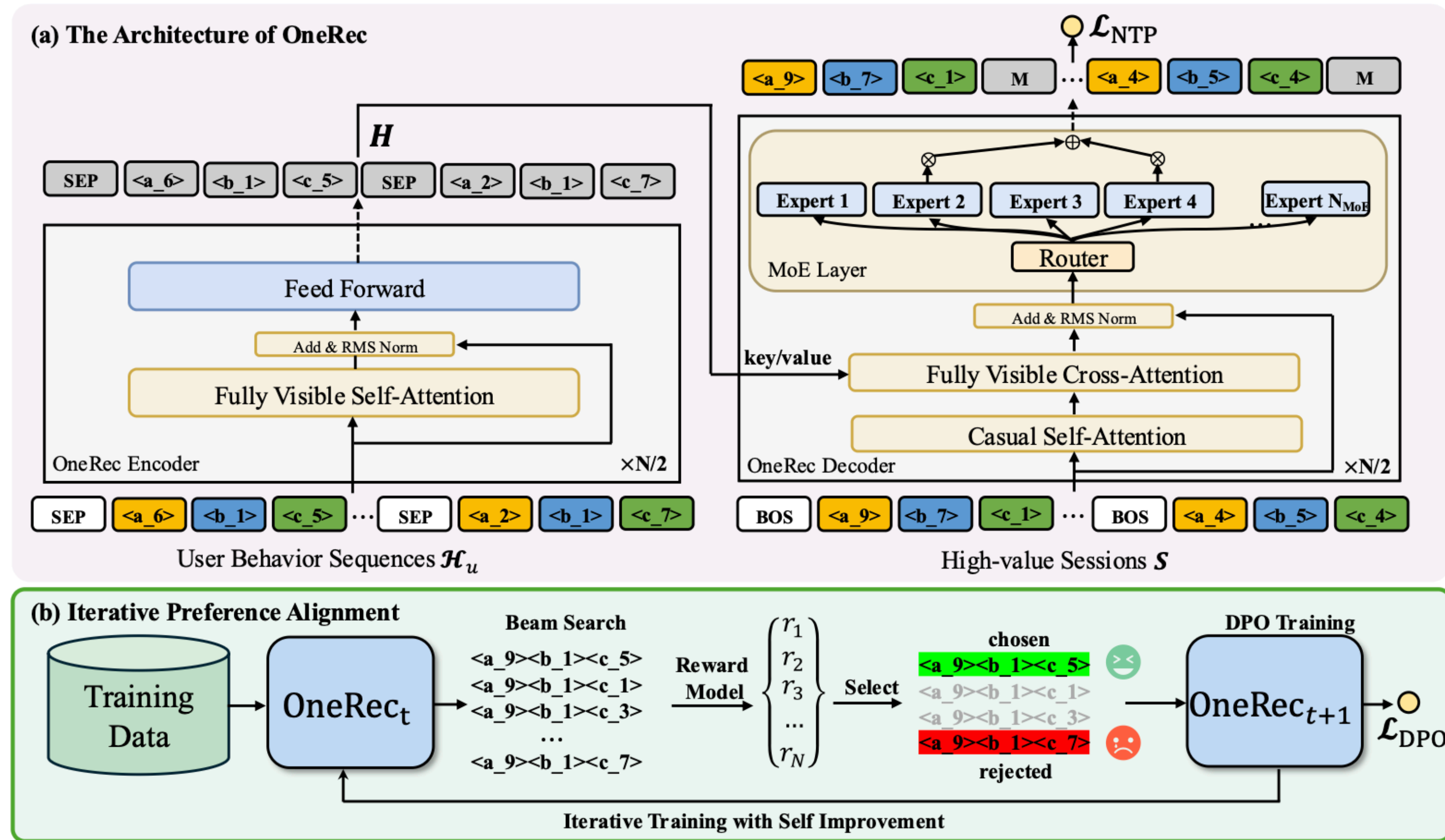


Figure 2: The overall framework of OneRec, consists of two stages: (i) the session training stage which train OneRec with session-wise data; (ii) the IPA stage which utilizes iterative direct preference optimization with self-hard negatives.

PLUM: LLM для рекомендаций

- arXiv:2510.07784, Google, октябрь 2025
- Pre-trained Language Models for industrial-scale recommendations
- Ключевая идея: взять готовую LLM (например, Gemini-nano), адаптировать под рекомендации
- Три этапа: item tokenization → continued pre-training → task fine-tuning
- Отличие от OneRec: используют knowledge из pre-trained LLM (мир, текст, semantic)
- pipeline "Gemini pre-trained → + SIDs → continued PT → recsys-model"

PLUM: LLM для рекомендаций

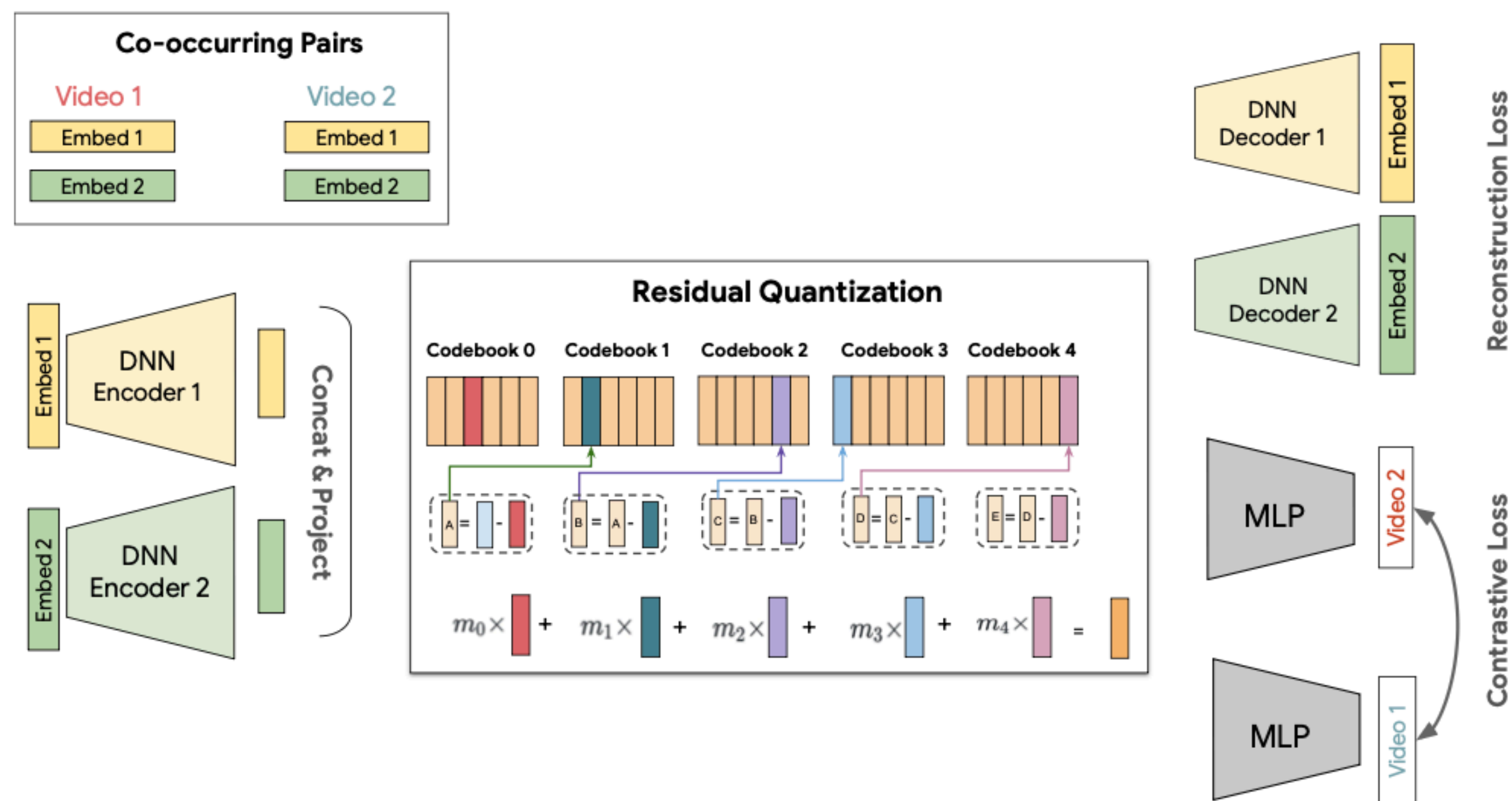


Figure 1: Illustration of our Semantic ID model. It takes two multi-modal video embeddings, encodes them, and compresses the result into a quantized ID using a residual quantizer. This ID is trained to both reconstruct the original inputs and semantically cluster co-occurring videos using a contrastive loss.

PLUM: LLM для рекомендаций

Как превратить LLM в рекоммендер

- Стадия 1: Item Tokenization — расширение словаря LLM Semantic ID токенами
- Стадия 2: Continued Pre-training — языковое моделирование на истории пользователей в формате "юзер смотрел X, Y, Z → следующее: W»
- Стадия 3: Task Fine-tuning — под конкретные поверхности (YouTube, Shopping, ...)
- Результат: SOTA на YouTube, обгоняет специализированные модели
- pre-trained general LLM → "rec-aware LLM" → "surface-specific model"

PLUM: LLM для рекомендаций

(a) Example 1	
	 (1/3) B6t2lY9sUxA Natural Remedies for Hypothyroidism and Hashimoto's Disease Dr. Josh Axe 556827 views  (2/3) BHEhxPuMmQl Physicist Brian Cox explains quantum physics in 22 minutes Big Think 2365348 views  (3/3) iNyUmbmQQZg Is it normal to talk to yourself? TED-Ed 8682099 views
Video Thumbnail	
Few-shot Input	The video A1991 ... H10 is about health and pain management. The video A364 ... H37 is about the basics of quantum physics. The video A37 ... H25 is about
LLM-Initialized CPT Output	psychology and mind
Randomly-Initialized CPT Output	100% video A252 H7

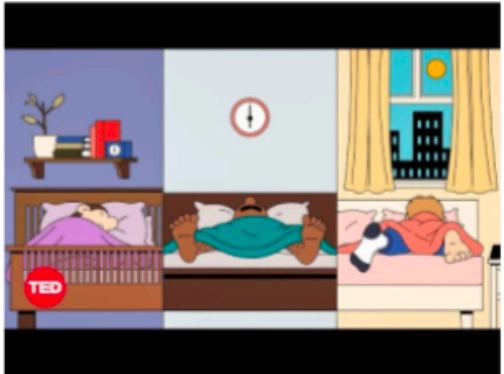
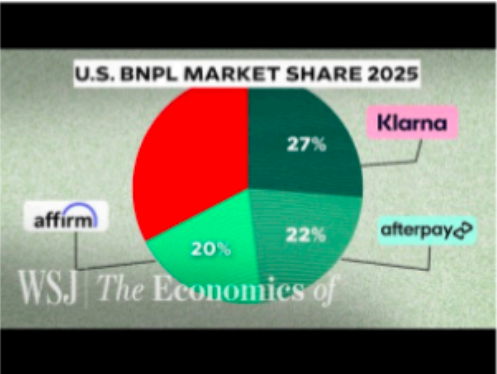
(b) Example 2	
	 (1/3) fQUeDdaVoWo Do You Really Need 8 Hours of Sleep Every Night? Body Stuff with Dr. Jen Gunter TED TED 4129771 views  (2/3) 3yC8WDjCIAU How 'Buy Now, Pay Later' Makes Billions From 'Free' Loans WSJ The Economics Of The Wall Street Journal 904680 views  (3/3) WSUj3PRvzzg Is being bilingual good for you brain? BBC Ideas BBC News 753351 views
Video Thumbnail	
Few-shot Input	Video A37 ... H41 answers the question: do you really need 8 hours of sleep? Video A1926 ... H8 answers the question: how buy-now pay-later loan business makes profit? Video A1882 ... H32 answers the question:
LLM-Initialized CPT Output	the impact of language in your life.
Randomly-Initialized CPT Output	- - - 100% -1999-11-16

Table 8: Qualitative examples of few-shot, in-context learning example. Each table shows the text-based prompt and model’s output from two different initializations. The thumbnails (top) show the visual content for each corresponding Semantic ID in the prompt.

Scaling laws

Scaling laws в рекомендациях: общая картина

Работают ли законы Kaplan-Hoffmann в recsys?

- Напоминание — два закона из мира LLM:
 - **Kaplan (2020)**: качество = степенной закон от N (параметры), D (данные), C (компьют). Вывод: растить модель быстрее данных
 - **Hoffmann / Chinchilla (2022)**: при правильной настройке N и D нужно растить **одинаково** — ~20 токенов на 1 параметр
- Что переносится в recsys:
 - Wukong (Meta), HSTU, TIGER демонстрируют степенные кривые $\text{loss} \leftrightarrow \text{масштаб}$
 - Для generative retrieval картина ближе всего к LLM

Scaling laws в рекомендациях: общая картина

Работают ли законы Kaplan-Hoffmann в recsys?

- HSTU (Meta): $\alpha \approx 0.15-0.2$, качество растёт от 1.5M до 1.5B
- ARGUS (Yandex): подтверждение до 1B параметров
- OneRec: scaling до 2.6B параметров в статье
- PLUM: переиспользование scaling уже обученной LLM
- Основной вывод: да, работают, но α меньше чем в NLP

Data scaling vs parameter scaling

Что важнее — данные или параметры?

- Chinchilla lesson переносится: оптимум при сбалансированном росте
- В recsys данных много (триллионы событий), но большинство шумные
- Качество данных критично: dedup, filter bots, quality signals
- Context length scaling: от 100 до 8000+ событий даёт прирост
- Временной контекст: "что юзер делал 2 года назад" всё ещё релевантно

Inference scaling: НОВЫЙ ВЫЗОВ

- Ограничения: rec-SLA ~50-100ms, vs LLM-chat ~секунды
- Решения:
 - Distillation (ARGUS: 1B teacher → 100M student)
 - KV-cache sharing между юзерами с общим префиксом
 - Batched serving (LiRank)
 - Speculative decoding для генеративных ретриверов
 - Quantization (INT8/INT4)
- Инженерный компромисс: качество vs стоимость GPU

ИТОГИ

Карта ландшафта 2023-2025

Работа	Год	Задача	Параметры	Ключевой вклад
TIGER	2023	Retrieval	~100M	Semantic IDs
HSTU	2024	Ranking + Retrieval	1.5B	Scaling laws, HSTU block
LiRank	2025	Ranking	~100M	Industrial multi-surface
OneRec	2025	Unified	2.6B	IPA, session generation
ARGUS	2025	User modeling	1B	Pre-train + distill in recsys
PLUM	2025	Unified	Gemini-size	LLM adaptation
RecGPT	2025	Retrieval	-	Instruction tuning

Вопросы